AISE 4010

Deep Learning for Time Series

September 9, 2026

Lecture 1:

Introduction to Time Series and Deep Learning

Outline

- Course Information
  - Instructor Information
  - Class Schedule
  - Evaluation
- Introducing deep learning in the context of time series
- Understanding the basic concepts of time series data

Instructor Information

- Contact:
  - Please contact me through OWL Messages
  - **Email:** snikan@uwo.ca
- Office: TEB 239

- Consultation hours:
  - Mondays: 11:30 am – 1:30 pm (by email appointment)

- Course Website https://owl.uwo.ca
  - All course information (e.g., assignments, tutorials…) will be posted on the website. Check the website regularly for course announcements.

Evaluation

Component	Weight	How it works
Lab assignments	15%	Python group work, marked in the lab. No take-home assignments.
Quizzes	5%	Mixed format: multiple choice, programming, written answer, …
Group project	45%	Three phases, each demonstrated in the lab, plus a final report.
Final exam	35%	In person, closed book, practical and theory. Regular exam period.

- ***You need to bring your own laptop to class for quizzes.***
- Bring your own laptop to every lab.
- Everything is posted on OWL — check it for announcements.

Course Roadmap

- Understand time-series data and their characteristics
- Foundational concepts in time series analysis
- Different types of time-series modelling
- How to apply statistical and machine learning (deep learning, more specifically) modelling to time-series data
- Deep learning and advanced models like LSTMs, CNNs, Transformers, hybrid models and Generative AI in the era of time-series
- Evaluate the strengths and weaknesses of the modelling techniques

What do you need in this course?

- Knowledge on statistics and machine learning

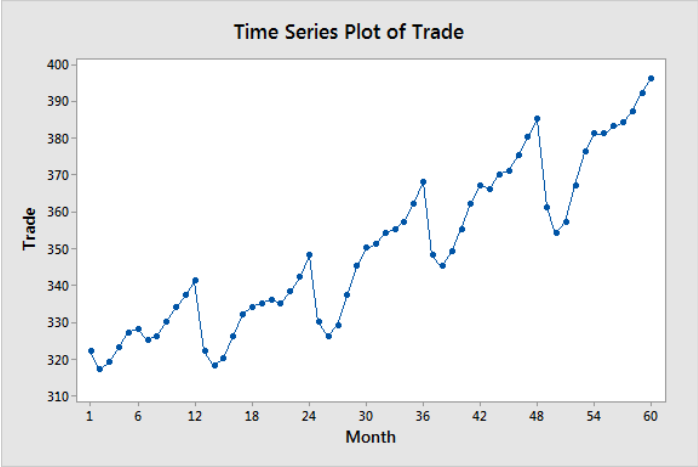
- Tools:
  - Python (know how to do programming)
  - Google Colab resources
  - Jupyter notebook environment
  - scikit-learn, PyTorch/TensorFlow open-source machine learning libraries and frameworks

1.1: Introduction to Time Series and Their Modelling Strategies

What is Time Series Data?

- A time series is a **discrete or continuous** sequence of observations that depends on time (data collected on the same metric/object at regular/irregular time intervals).

- The time period may be measured in years, seasons, months, days, hours, minutes, seconds or any other suitable unit of time.
  - This data can be anything from stock prices and weather reports to sensor readings in a factory and network traffic.
  - The key characteristic: each data point is **dependent on time**, making the analysis unique from other types of data.
  - The temporal dependency often makes past values influence future values. Understanding these dependencies is crucial for building effective predictive models.
  - Predicting future behaviors will allow organizations to adapt to future conditions.



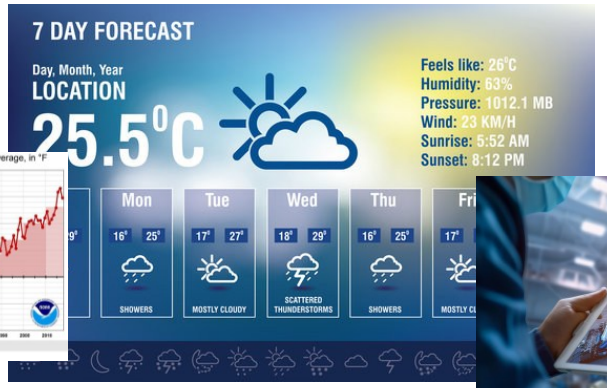
Applications

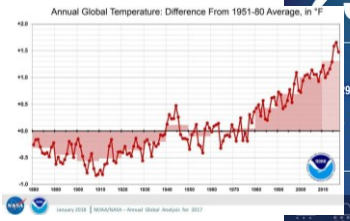
- Finance
- Healthcare
- Weather forecast
- Energy and smart grid
- Internet of Things (IoT)
- Manufacturing …



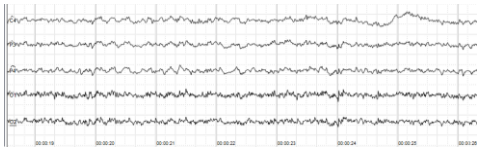
















Different Time Series Analysis

As continuous monitoring and data collection become more common, the need for competent time series analysis will increase.

Time series analysis asks one question in several forms: how does this variable behave over time? how much does its past tell you about what comes next?

The main tasks:

- Forecasting
  - aims to predict future values based on past data
- Classification & Clustering
  - involves categorizing time series data based on their characteristics
- Anomaly detection
  - aims to identify unusual or unexpected patterns in the data
- Generation
  - synthesise new series with the same statistical structure

Time Series Modelling

- Classical statistical modelling

  (parametric models on lags with interpretable coefficients e.g., AR/MA/ARMA/ARIMA, SARIMA)

- State-space & Bayesian

  (e.g., Kalman filtering/smoothing, missing data handling).

- Feature-based Machine Learning

  (turn series into a supervised table and fit a regressor/classifier, *e.g.*, Random Forest, XGBoost, LightGBM, SVR, k-NN)

- **Deep learning**

  (learn representations directly from sequences, capture nonlinear/long-range structure. Use when: Large data, multivariate sensors, and complex patterns, e.g., RNN/GRU/LSTM, CNN/TCN, Transformers, N-BEATS, DeepAR/DeepVAR, TimeGPT)

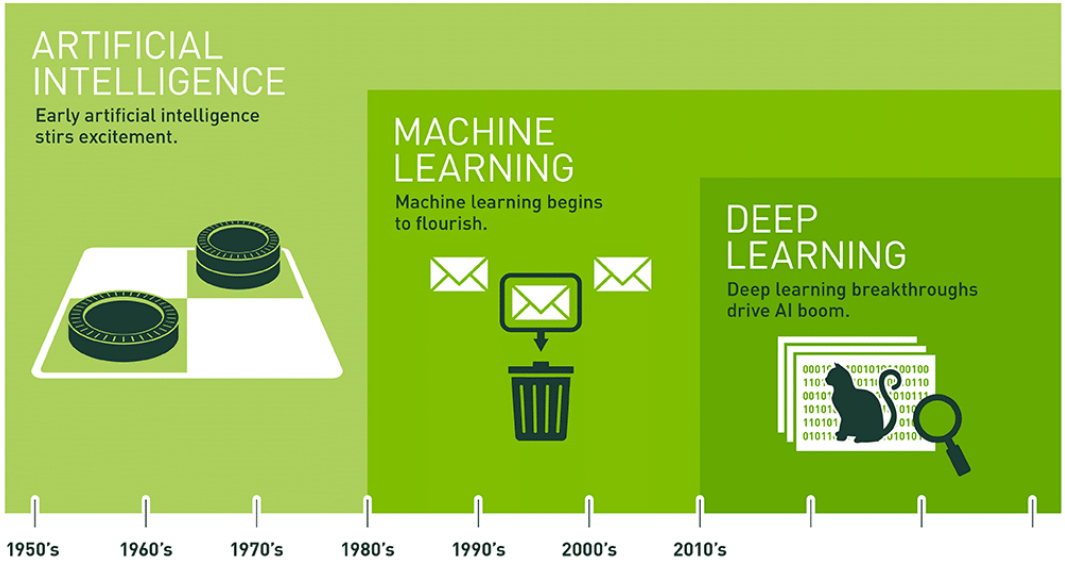
- Hybrid & ensembles

  (combine strengths with model stacking, when you want more robustness)

Recap: Machine and Deep Learning

- Machine learning (ML) finds the rules for a task from examples, instead of being given the rules.
- Deep learning (DL) is ML with many layers of representation, learned jointly from raw data rather than hand-engineered.

- For time series this matters because the features that classical methods make you specify, such as "which lags", "which seasonal terms", "which interactions", are learned automatically instead.
  - The price is data. Deep models need considerably more data than a statistical or ML model (e.g., ARIMA or a gradient-boosted table of lags).



Why Deep Learning for Time Series?

- Time series are complex, high-dimensional, large-scale, and intricate temporal dependencies that traditional methods struggle with.
- Deep learning are effective in capturing complex patterns in data
  - (e.g., remarkable in image recognition, handwriting transcription, autonomous driving, machine translation, text-to-speech conversion, text-to-image/video generation, digital assistants, ad targeting, etc.)
- DL automates the most crucial step of feature engineering in ML/shallow learning:
  - incrementally (layer-by-layer), where increasingly complex representations are developed and learned jointly.
- DL handles nonlinearity & interactions
- It captures long-range dependencies

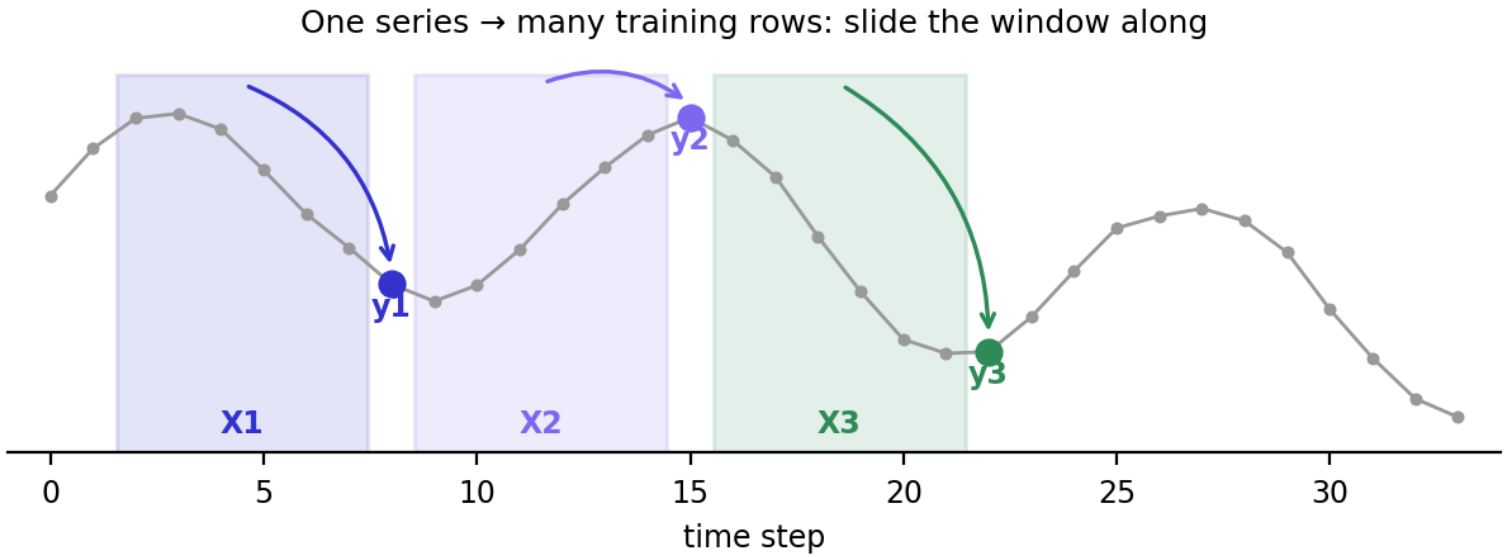
Therefore, enabling us to:

- make accurate predictions
- handle anomalies
- extract valuable insights



Building a Supervised Dataset

- A time series is not a supervised dataset. You have to build one from it.
- Take a window of L past values as the input, and the value h steps ahead as the target. Slide the window forward one step and you have another row.
  - Input X: the last L observations, plus anything else known at that moment (calendar, other sensors).
  - Target y: the observation h steps later.
  - Objective: an error measured on values the model has not seen.



- Every model in this course — MLP, CNN, RNN, Transformer — consumes exactly this X and y. Only the mapping changes.

Example: Building X and y

- Eight observations. Lookback L = 3, horizon h = 1.

y = [12, 15, 14, 18, 21, 19, 24, 26]

Row	X (three past values)	y (next value)
1	12, 15, 14	18
2	15, 14, 18	21
3	14, 18, 21	19
4	18, 21, 19	24
5	21, 19, 24	26

- Eight numbers became five training rows.

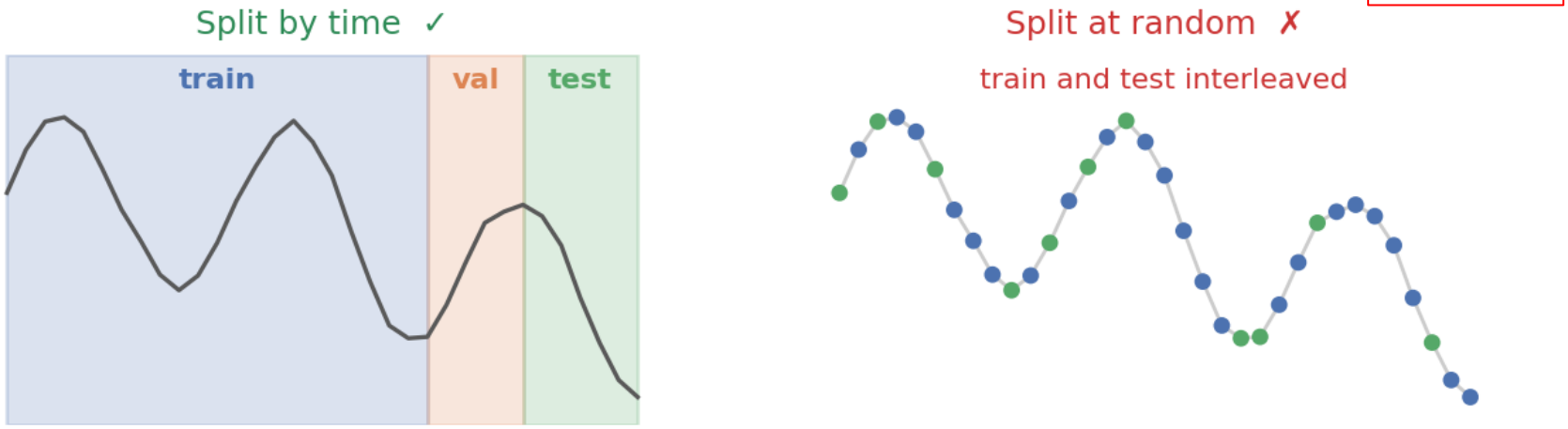
$$rows = n - L - h + 1$$
$$= 8 - 3 - 1 + 1 = 5$$

- A short window and a long horizon cost you rows. On a small dataset that is often the binding constraint.
- Nothing after the last observation is used (your model learns from this how to map past to future).
- Then what your model produces after the last observation is a future forecast.

Two Decisions and One Rule

- Two decisions you make for every model in this course:
  - Lookback L — how much history goes in. Too short and the window cannot contain the season; too long and most of the inputs are noise. Start at one or two seasonal periods and test.
  - Horizon h — how far ahead. Predicting all h steps at once is direct. Feeding predictions back in is recursive, and its errors compound.
- And one rule that is not negotiable: split by time.

leakage



A random split puts tomorrow in the training set. Every score after that is wrong in a flattering direction.

1.2: Basic Concepts of Time Series Data

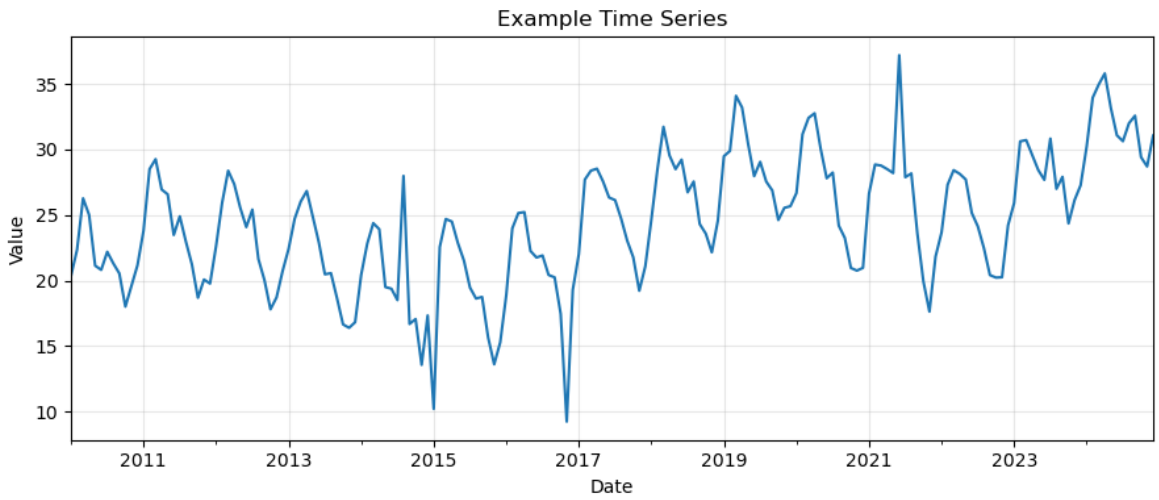
Time Series Data

- Let's plot a time series data:

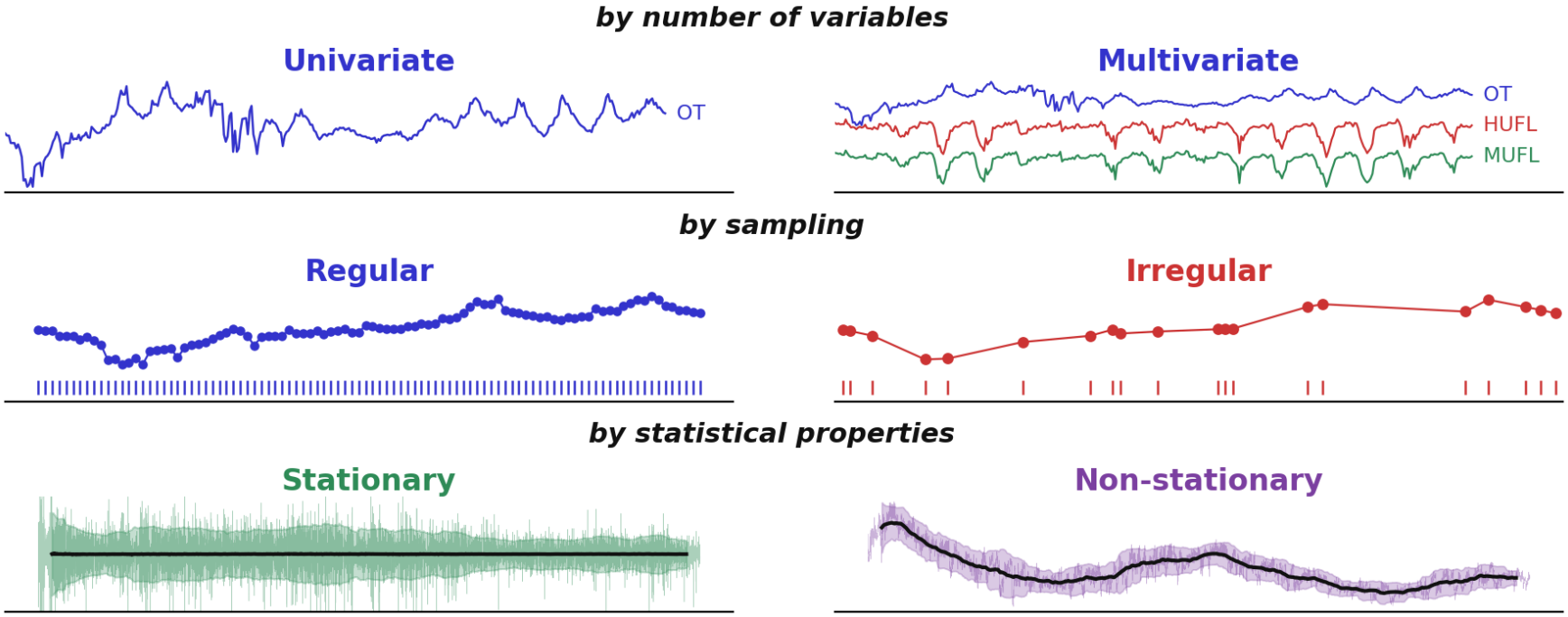
date	F1	F2	F3	F4	observed
2010-01-01	0	0	0	0.36566	20.36566
2010-02-01	0.02025	3.299038	0.275181	-1.24798	22.34649
2010-03-01	0.041	4.76314	0.575172	0.900541	26.27985
2010-04-01	0.06225	4	0.895647	0.051562	25.00946
2010-05-01	0.084	2.165064	1.230581	-2.34124	21.1384

```python
import pandas as pd
import matplotlib.pyplot as plt

# Load monthly series
df = pd.read_csv("example_data.csv",
                 parse_dates=["date"], index_col="date")
# Basic line plot
ax = df["observed"].plot(figsize=(9,4))
ax.set_title("Example Time Series")
ax.set_xlabel("Date"); ax.set_ylabel("Value"); ax.grid(True,
plt.tight_layout(); plt.show()
```



Types of Time Series Data

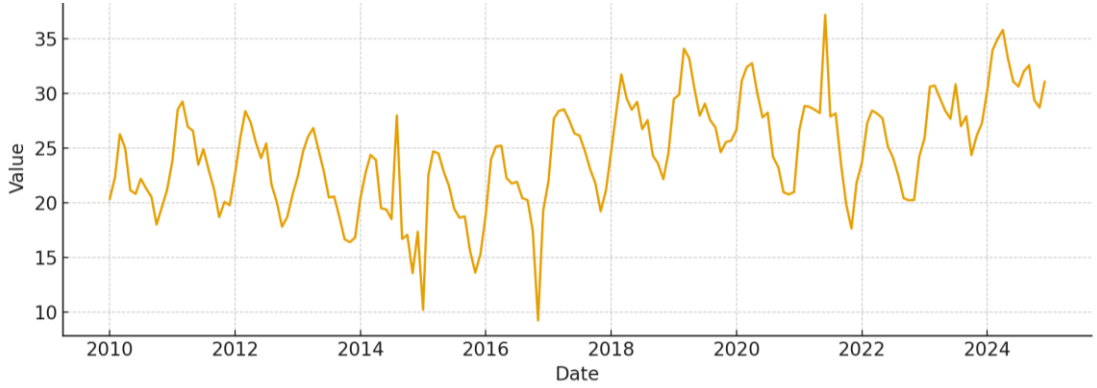


- By number of variables
  - Univariate — one variable over time; a single column of numbers (Oil Temperature).
  - Multivariate — several variables on one clock, each able to help predict the others (Oil Temperature, High UseFul Load, Middle UseFul Load).
- By sampling
  - Regular — a fixed step between observations. Every windowing method assumes this.
  - Irregular — uneven gaps, so "24 steps back" is no longer a day.
- By statistical properties
  - Stationary — mean, variance and autocovariance stay roughly constant.
  - Non-stationary — statistical properties drift. Most real series do. Classical models need it removed.

https://github.com/zhouhaoyi/ETDataset

Components of Time Series Data

- Time series data may exhibit trends — long-term movement/direction upward or downward.
- There might be seasonality — repeating/recurring patterns at regular intervals (e.g., monthly, yearly).
- There can also be cyclic patterns that occur over irregular intervals (over longer periods, beyond seasonality).
- Also, irregularity (or noise/random variation) that are short-term, unpredictable variations or random fluctuations in the data that cannot be easily explained.



Each type of pattern requires specific analytical approaches

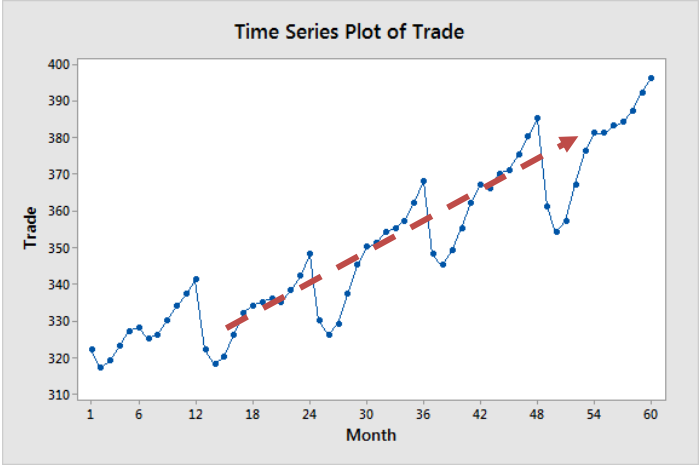
Recognizing these patterns is key to effective analysis.

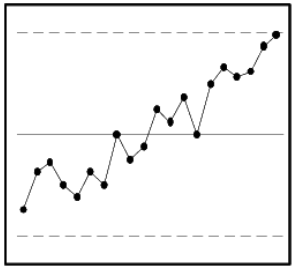
Trend

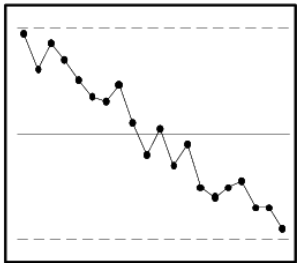
A pattern in data that shows the movement of a series to relatively higher or lower values over a long period of time.

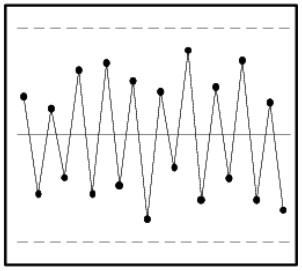
It's the long-term systematic change in the series level (upward/downward drift

- Upward/Increasing
- Downward/Decreasing
- No trend



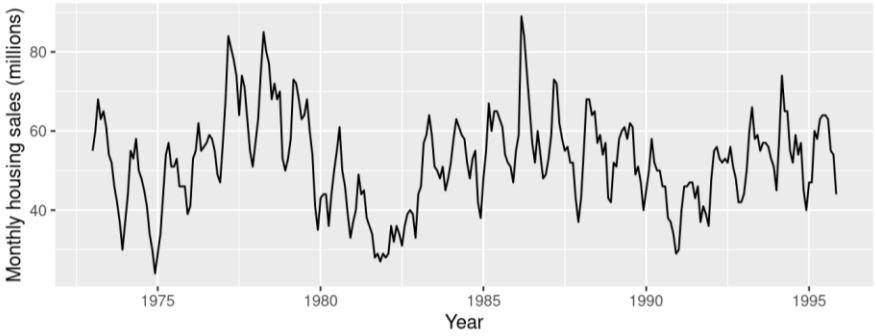


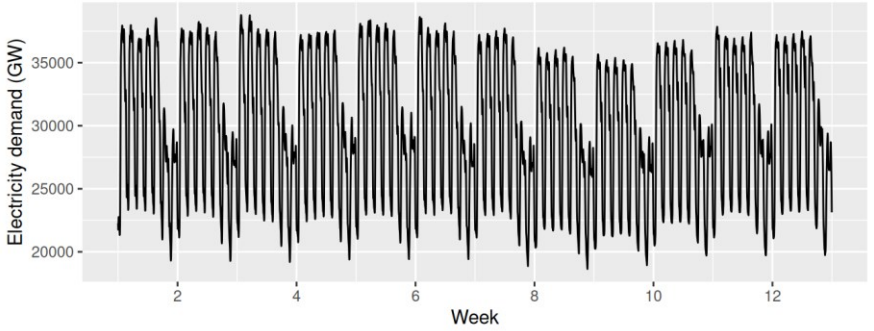




Seasonal Time Series

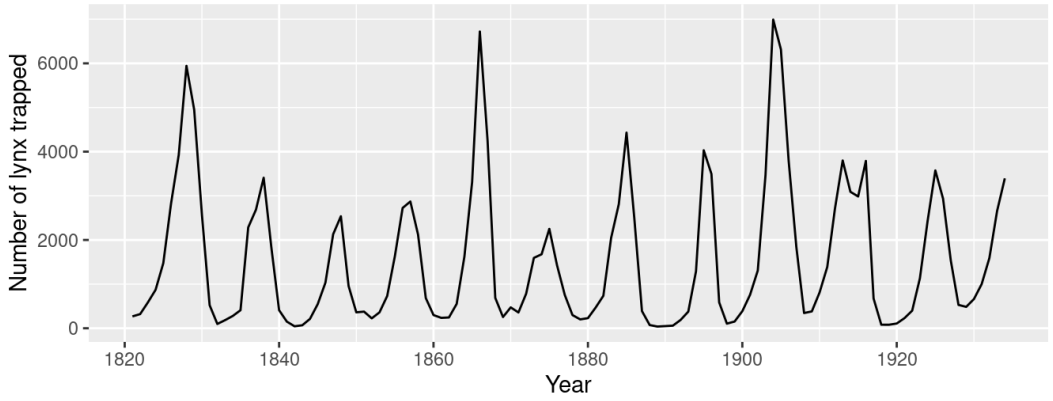
- A seasonal pattern exists when a series is influenced by seasonal factors (e.g., the quarter of the year, the month, or day of the week).
- Seasonality is always of a fixed and known period (associated with some aspect of the calendar).
- Hence, seasonal time series are sometimes called periodic time series.





Cyclic time series

- A cyclic pattern exists when data exhibit fluctuations (rises and falls) that are not of fixed period. The duration of these fluctuations is usually of at least 2 years.
- Cycles – no fixed interval of time series behaviors and patterns
- Think of business cycles which usually last several years, but where the length of the current cycle is unknown beforehand.



How to Estimate the Trend?

Simple ways to estimate trend:

- Simple Moving average (SMA): robust & simple
- Exponential moving average (EMA): reacts faster
- Decomposition (classical or STL): isolates a smooth "trend" component

Other Types of Moving Averages in Time Series Analysis			
Moving Average Type	Formula / Mathematical Definition	Weight Distribution	Best Used For
Simple Moving Average (SMA)	$\mathrm{SMA}_t = \frac{1}{k}\sum_{i=0}^{k-1} X_{t-i}$	Equal weighting across all points in the sliding window.	General trend visual analysis, baseline smoothing, and low-volatility tracking.
Weighted Moving Average (WMA)	$\mathrm{WMA}_t = \frac{\sum_{i=0}^{k-1} w_i \cdot X_{t-i}}{\sum_{i=0}^{k-1} w_i}$	Linearly decreasing weights assigned to older observations.	Scenarios where recent observations carry higher predictive value than historical ones.
Exponential Moving Average (EMA)	$\mathrm{EMA}_t = \alpha X_t + (1-\alpha)\mathrm{EMA}_{t-1}$	Exponentially decaying weights (higher weight on most recent points).	Rapidly changing systems, technical trading indicators, and real-time signals.
Centered Moving Average (CMA)	$\mathrm{CMA}_t = \frac{1}{2m+1}\sum_{i=-m}^{m} X_{t+i}$	Symmetric weighting using observations both before and after time t.	Historical classical decomposition (extracting trend-cycle & seasonality offline).

Simple Moving Average (SMA) Smoothing

- The classical method of time series decomposition originated in the 1920s.
- Moving Average (MA) in time series analysis calculates the mean of data points within a sliding window across successive time intervals.
- It still forms the basis of many time series decomposition methods.
- A moving average of order m $(m - MA)$ can be written as:

$$\hat{T}_t = \frac{1}{m} \sum_{j=-k}^{k} y_{t+j}\,, \qquad where\ m = 2k + 1$$

- This estimate of the trend-cycle at time t
- It can be used for trend identification, noise reduction, seasonality extraction, baseline forecasting and feature engineering

Simple Moving Average (SMA) Smoothing

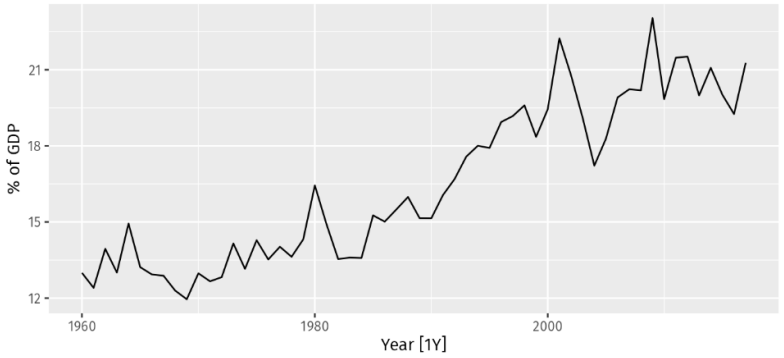
- Example1:

Time series of annual Australian exports of goods and services: 1960–2017.

- 5-year moving average (5-MA) for the year 1962
- k (number of years before and after the target year to include in the average)

$k = 2, \qquad m = 5$

Year	Exports	5-MA
1960	12.99	
1961	12.40	
1962	13.94	13.46
1963	13.01	13.50
1964	14.94	13.61
1965	13.22	13.40
1966	12.93	13.25
1967	12.88	12.66
...	...	...
2010	19.84	21.21
2011	21.47	21.17
2012	21.52	20.78
2013	19.99	20.81
2014	21.08	20.37
2015	20.01	20.32
2016	19.25	
2017	21.27	



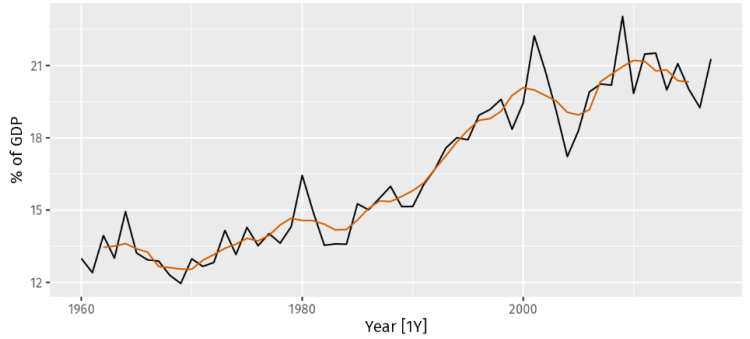
$\hat{T}_3$: *smoothed value*

$$= \frac{12.99 + 12.40 + 13.94 + 13.01 + 14.94}{5}$$

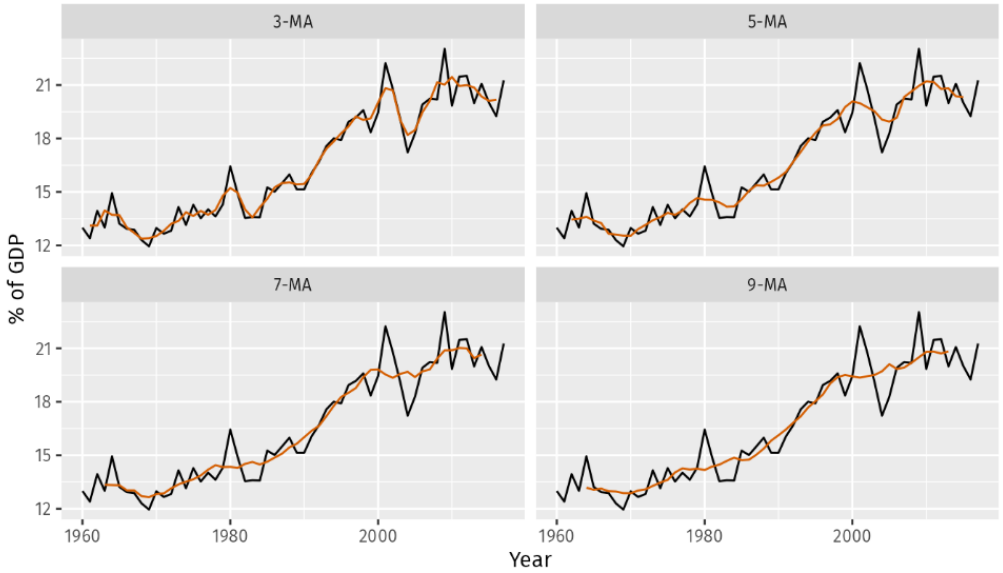
Simple Moving Average (SMA) Smoothing

- Example1:

Trend-cycle (in orange) is smoother than the original data and captures the main movement of the time series.



The order of the moving average determines the smoothness of the trend-cycle estimate



Simple Moving Average (SMA)

- Example2:

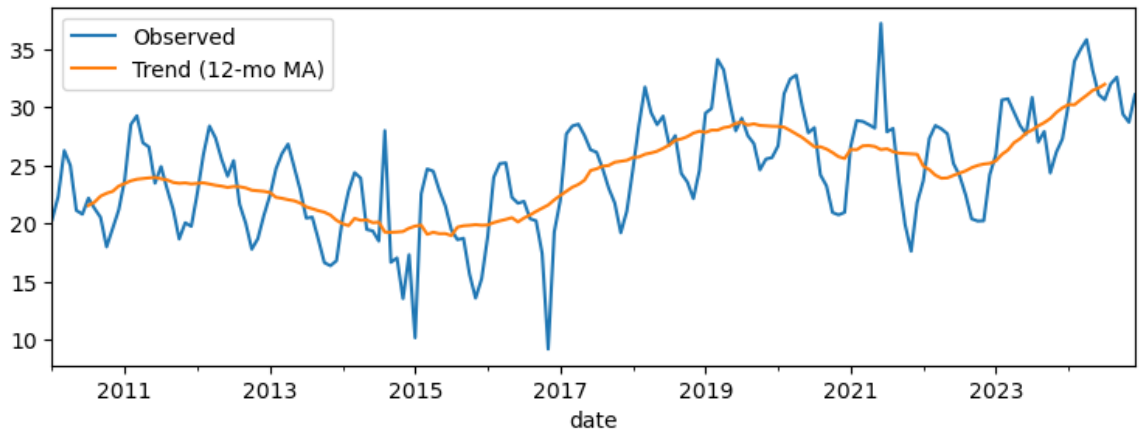
```
import pandas as pd
import numpy as np
from statsmodels.nonparametric.smoothers_lowess import lowess

df = pd.read_csv("example_data.csv",
                 parse_dates=["date"], index_col="date")

# 1) Centered 12-month moving average (classic trend estimate)
trend_ma12 = df["observed"].rolling(window=12, center=True).mean()

# Detrend (additive)
detrended = df["observed"] - trend_ma12

# Quick peek (one line plot at a time in class)
ax = df["observed"].plot(figsize=(9,3), label="Observed")
trend_ma12.plot(ax=ax, label="Trend (12-mo MA)")
ax.legend()
```



Simple Moving Average (SMA)

- If the order is an even number (e.g., quarterly, monthly repeated patterns/seasonality in data), we apply a moving average to a moving average to center it. For example, we might take a moving average of order 4, and then apply another moving average of order 2 to the results.

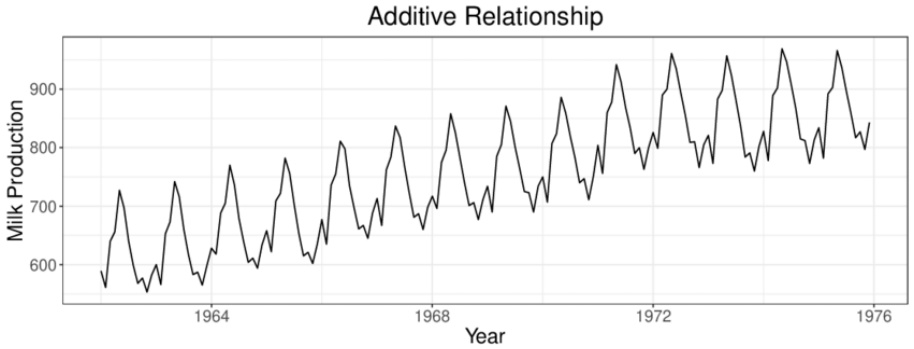
Quarter	Beer	4-MA	2x4-MA
1992 Q1	443.00		
1992 Q2	410.00	451.25	
1992 Q3	420.00	448.75	450.00
1992 Q4	532.00	451.50	450.12
1993 Q1	433.00	449.00	450.25
1993 Q2	421.00	444.00	446.50
...	...	...	...
2009 Q1	415.00	430.00	428.88
2009 Q2	398.00	430.00	430.00
2009 Q3	419.00	429.75	429.88
2009 Q4	488.00	423.75	426.75
2010 Q1	414.00		
2010 Q2	374.00		

$$\hat{T}_t = \frac{1}{2}\left[\frac{1}{4}(y_{t-2} + y_{t-1} + y_t + y_{t+1}) + \frac{1}{4}(y_{t-1} + y_t + y_{t+1} + y_{t+2})\right]$$
$$= \frac{1}{8}y_{t-2} + \frac{1}{4}y_{t-1} + \frac{1}{4}y_t + \frac{1}{4}y_{t+1} + \frac{1}{8}y_{t+2}.$$

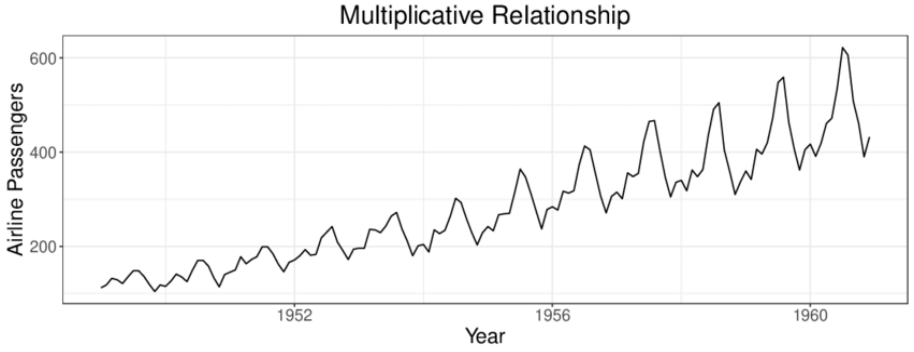
Note: If we use the MA with an order equal to the order of the length of the season (e.g., a 12-month window for monthly data or a 7-day window for daily data), we end up removing the seasonal component

Additive vs Multiplicative Models

- Additive and multiplicative models are two approaches to decomposing time series data into components like trend, seasonality, and residuals.
- The key difference lies in how these components interact.
- Additive models assume they sum together ($y_t = T_t + S_t + R_t$)
  - Works best when the magnitude of seasonal fluctuations or residuals remains constant over time, regardless of the trend's growth or decline.



- Multiplicative models assume they multiply ($y_t = T_t . S_t . R_t$).
  - Suitable when seasonal or residual effects scale with the trend.



Classical Decomposition

- Additive decomposition suits a series whose seasonal swing stays the same size as the level changes. Components are subtracted.
  1. Compute the trend-cycle component $\hat{T}_t$ using moving average (MA).
  2. Calculate the detrended series $y_t - \hat{T}_t$
  3. To estimate the seasonal component for each season, simply average the detrended values for that season. For example, with monthly data, the seasonal component for March is the average of all the detrended March values in the data. These seasonal component $\hat{S}_t$ is obtained by stringing together these monthly values.
  4. Residual/remainder component (random noise): **SUBTRACT** the trend-cycle and seasonal components from the series $R_t = y_t - \hat{T}_t - \hat{S}_t$

- Multiplicative decomposition suits a series whose seasonal swing grows with the level. The series is divided by each component.
  1. Compute the trend-cycle component $\hat{T}_t$ using moving average (MA).
  2. Calculate the detrended series $y_t/\hat{T}_t$
  3. To estimate the seasonal component for each season, simply average the detrended values for that season. For example, with monthly data, the seasonal component for March is the average of all the detrended March values in the data. These seasonal component $\hat{S}_t$ is obtained by stringing together these monthly values.
  4. Residual/remainder component (random noise): **DIVIDE** the series by the trend-cycle and seasonal components $R_t = y_t/(\hat{T}_t\hat{S}_t)$

Classical Decomposition

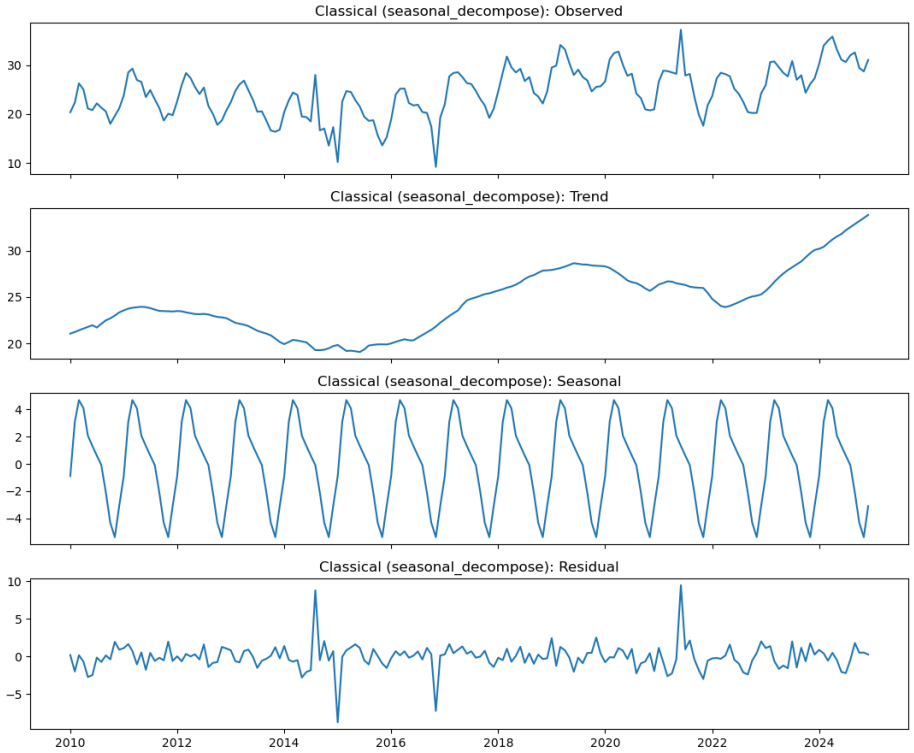
- Example:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

df = pd.read_csv("example_data.csv",
                 parse_dates=["date"], index_col="date")
y=df["observed"]
# Classical (additive or multiplicative)
class_res = seasonal_decompose(y, model="additive", period=12,
                               extrapolate_trend=12)

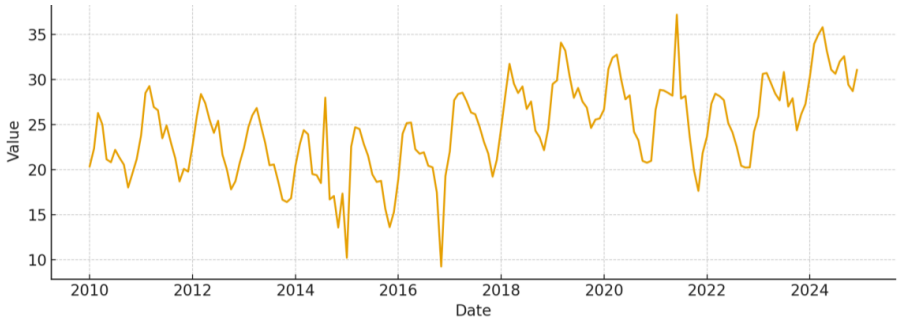
# Plot decomposition results
def plot_decomp(res, title_prefix):
    fig, axes = plt.subplots(4, 1, figsize=(11, 9), sharex=True)
    axes[0].plot(res.observed); axes[0].set_title(f"{title_prefix}: Observed")
    axes[1].plot(res.trend);    axes[1].set_title(f"{title_prefix}: Trend")
    axes[2].plot(res.seasonal); axes[2].set_title(f"{title_prefix}: Seasonal")
    axes[3].plot(res.resid);    axes[3].set_title(f"{title_prefix}: Residual")
    plt.tight_layout(); plt.show()

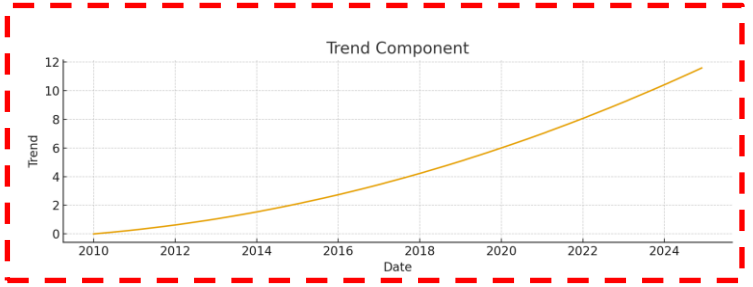
plot_decomp(class_res, "Classical (seasonal_decompose)")
```

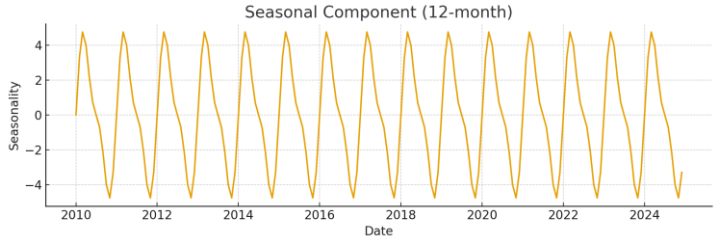


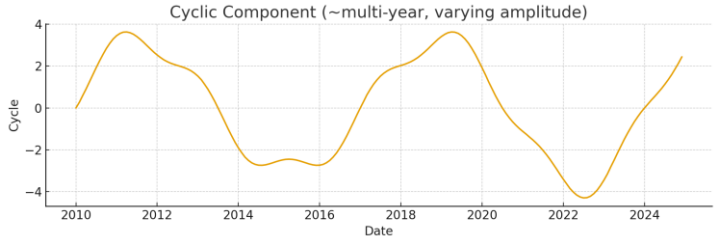
Estimating the Trend

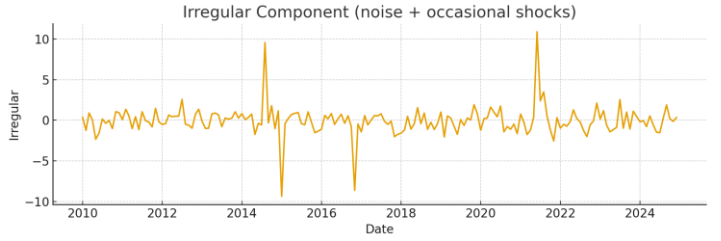
- Extracting the trend of time series using decomposition.





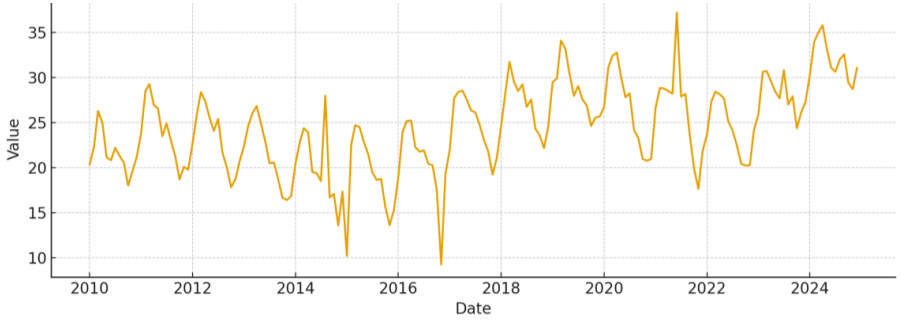


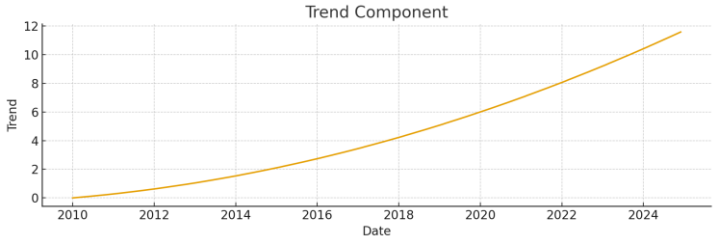


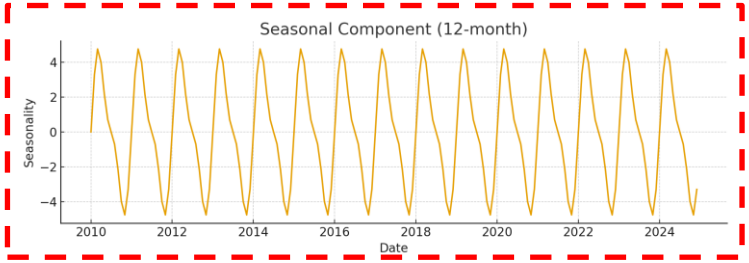


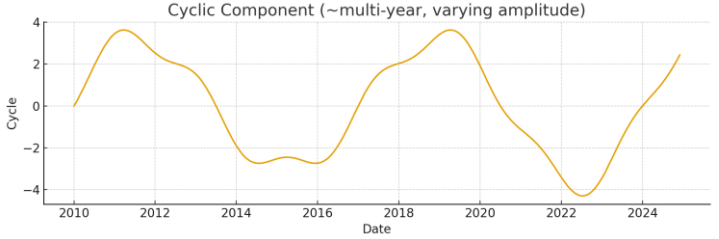
Estimating the Seasonality

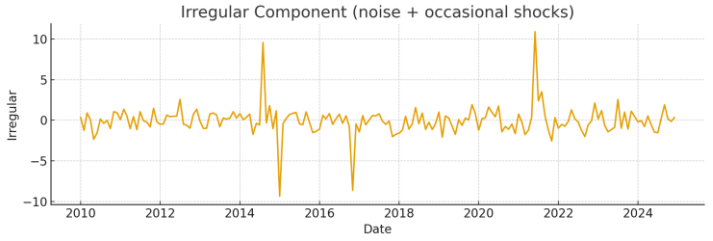
- Extracting seasonality using decomposition.





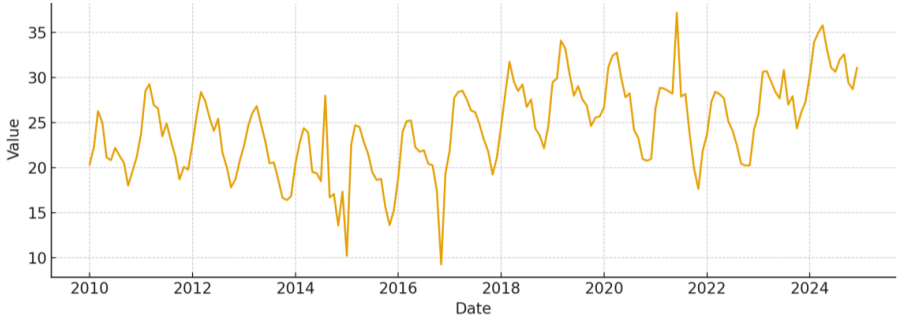


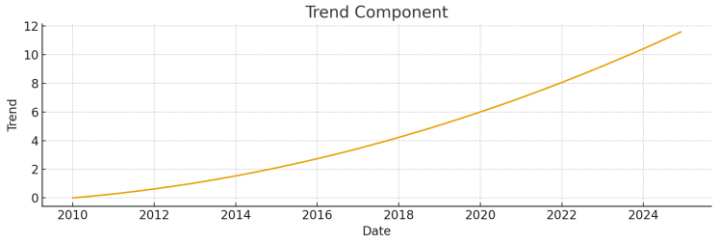


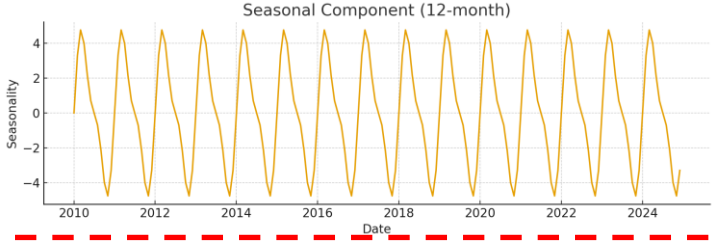


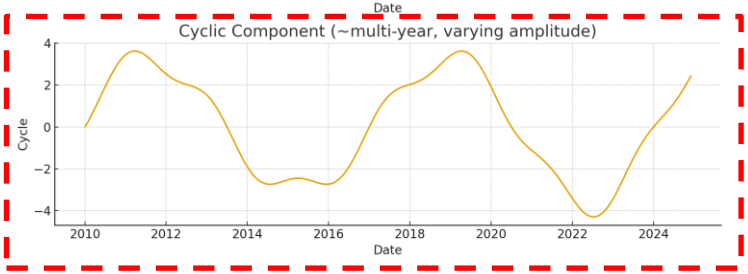
Estimating the Cycle

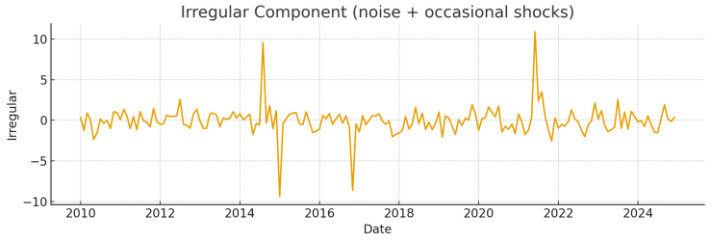
- Isolating cyclic pattern using decomposition.





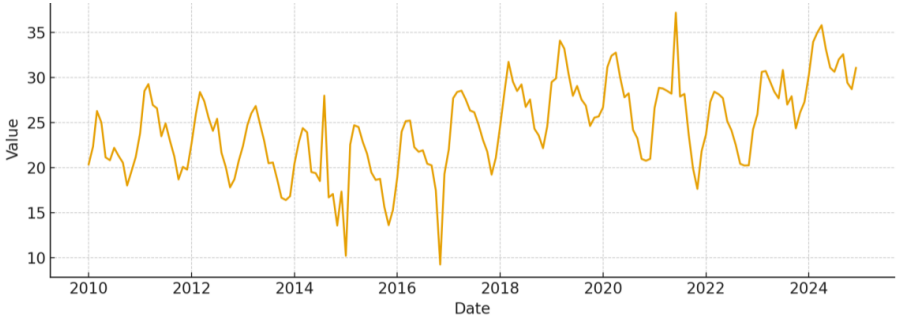


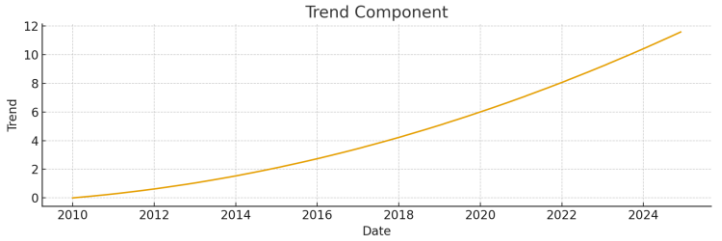


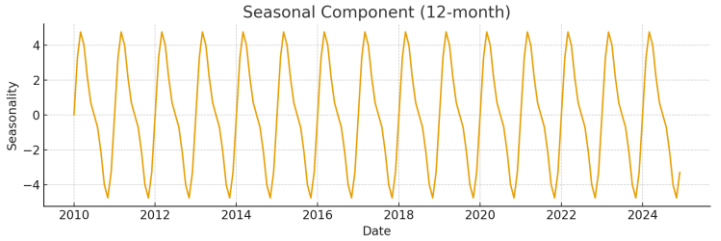


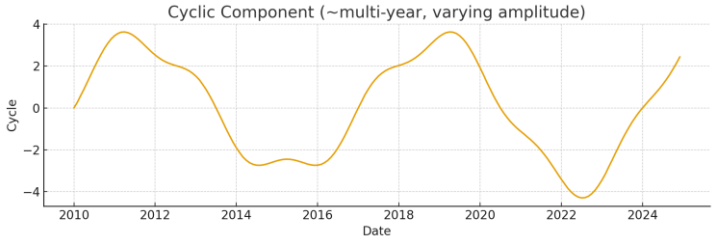
Estimating the Irregularity/Noise

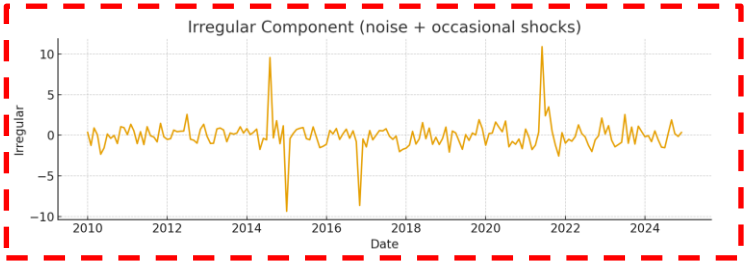
- Unexpected situations and events that cause huge time-series behavioral changes
- Isolating irregularity using decomposition.









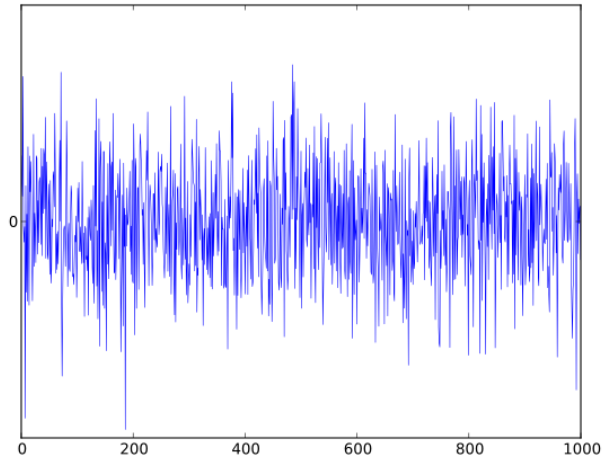


Stationary vs Non-stationary

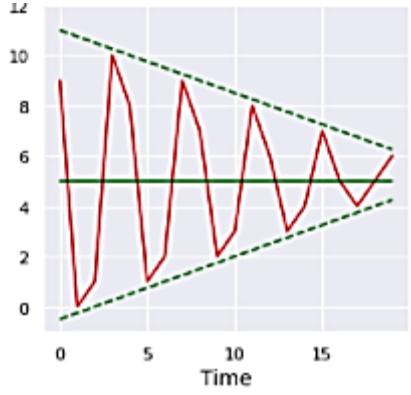
A time series is stationary when it has fairly stable statistical properties over time (time-invariant), particularly with respect to mean, variance and autocovariance

- Constant Mean
- Constant Variance
- No trend/seasonality

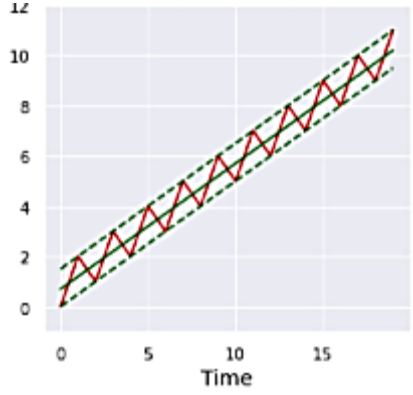
Many traditional statistical time series models stationary.



White Noise (stationary)



constant mean but non-constant variance (non-stationary)



constant variance but non-constant mean (non-stationary)

Autocovariance and Autocorrelation

- Autocovariance measures the linear relationship between different points in a single time series by calculating the covariance of the series with itself at a specific time lag of k

Lag: The time difference between two points in a time series.

- Autocovariance at lag k: $\gamma(k) = \mathrm{Cov}(y_t,\ y_{t-k})$

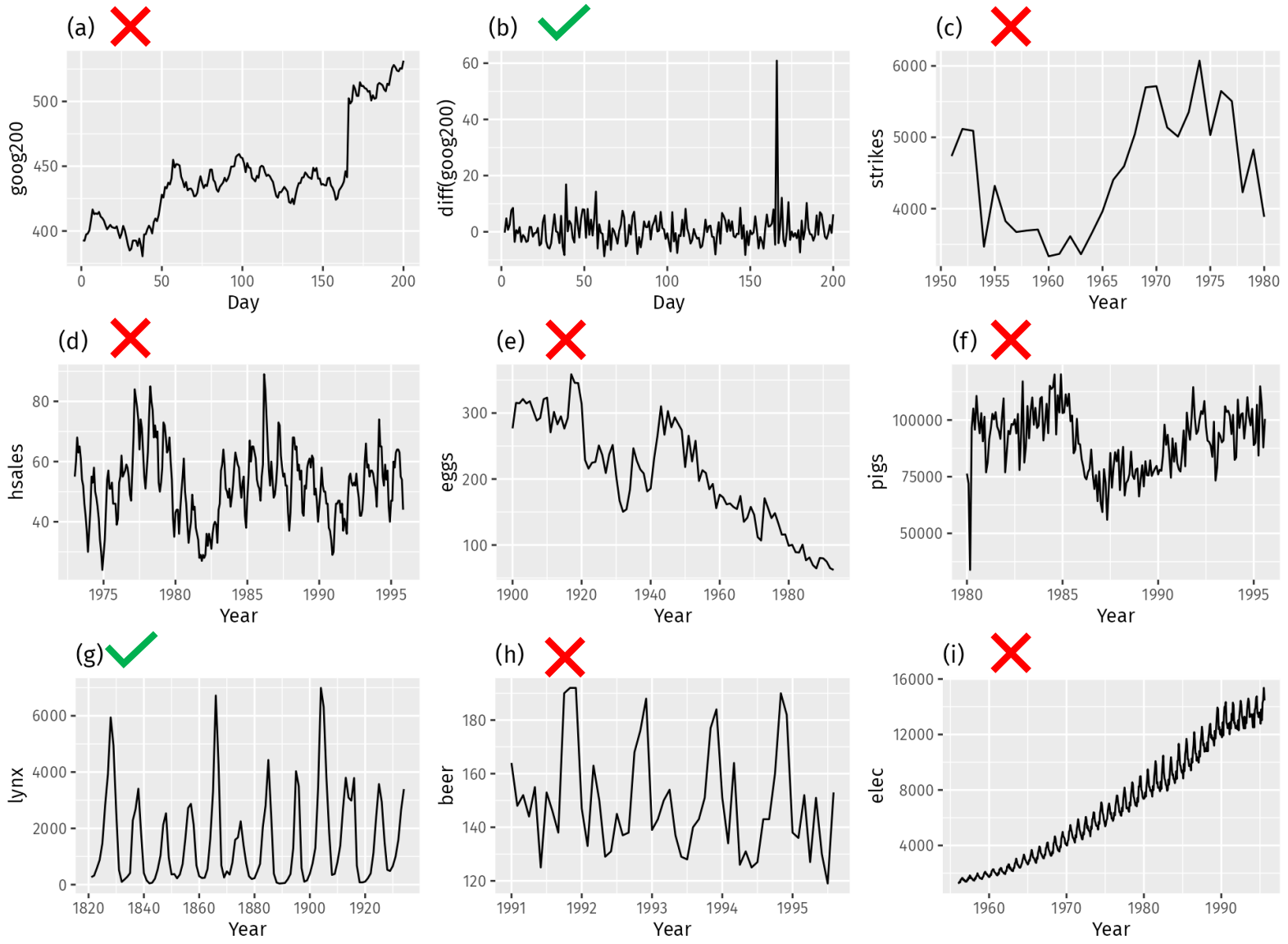
- It quantifies how much a time series' values at one point in time y_t are influenced by its values at earlier points in time y_{t-k} .

Note: Autocorrelation normalizes the autocovariance by the variance of the series, making it a dimensionless value between -1 and 1, which is easier to interpret as a correlation coefficient.

Autocorrelation at lag k: $\rho(k) = \dfrac{\gamma(k)}{\gamma(0)}$

More Examples

Stationary ✓
Non Stationary ✗



Challenges in Time Series Analysis

To analyse time series data successfully, several challenges must be addressed:

- Non-stationarity
  - occurs when the statistical properties of the data change over time, which can complicate analysis and modelling
- Missing data
  - no value is stored for an observation — faulty equipment, lost records, a dropped packet. It breaks the fixed time step that every windowing method depends on.
- Noise
  - Random, unpredictable variation that no model should be asked to fit. Fitting it is overfitting.
- High dimensionality
  - many sensors, few usable rows

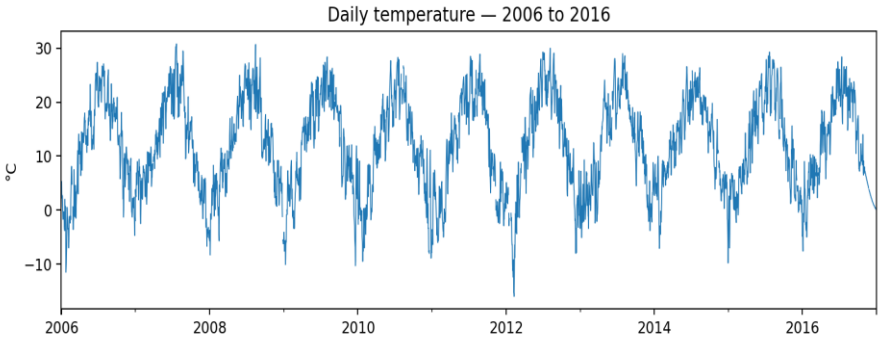
Data Preprocessing for Time Series

- Preprocessing and feature engineering methods are critical steps in time series analysis that can address the challenges, leading to more accurate analysis and modeling.
  - Handling missing values (imputation methods, such as mean substitution, forward-fill or time-based interpolation for short gaps — mean substitution is almost always wrong because it flattens the series)
  - Normalizing the data (to ensure all features are on the same scale — fit the scaler on the training split only, using the whole series leaks the future)
  - Smoothing to reduce noise (such as moving average or exponential smoothing)
  - Transformation can be applied to non-stationary data such as differencing ($y(t) - y(t-1)$), log operation, detrending, and seasonal adjustment.

1.3: Coding Example

Turn a CSV into a time series

- The timestamp is part of the data model — validate it before analysis.
  - Sorted, no duplicate dates.
  - One step size — 1 day, 4018 times. A perfectly regular daily grid with no absent dates.
  - But 47 temperatures are missing.
    - A regular index says nothing about whether the values are there. Two different questions, two separate checks.



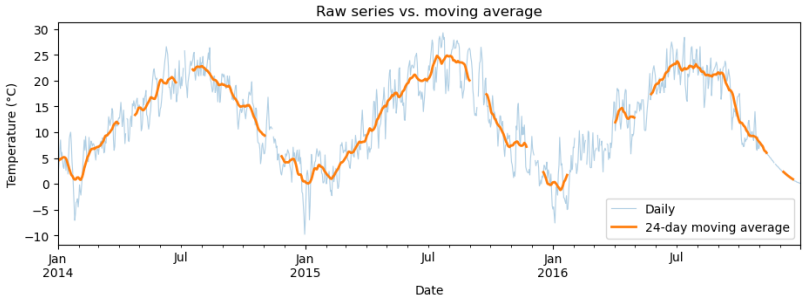

```
weather = pd.read_csv("weather.csv")
weather["Formatted Date"] = pd.to_datetime(
        weather["Formatted Date"])
weather = weather.sort_values("Formatted Date")
weather = weather.set_index("Formatted Date")

idx = weather.index
print(idx.is_monotonic_increasing)   # sorted?
print(idx.duplicated().sum())        # duplicates?
print(idx.to_series().diff()
         .value_counts().head())     # step size?
print(weather.isna().sum())          # missing VALUES?
```

Before any model, check time order, sampling interval, missing timestamps, and the physical meaning of one row.

Reveal trend and seasonality

- Use simple transformations to see patterns before choosing a model.
  - rolling(24) defaults to min_periods = 24.
  - 47 missing values → some of the smoothed line disappears.




```
# Smooth short-term noise to expose the longer pattern
trend_24 = temp.rolling(24, center=True).mean()

fig, ax = plt.subplots(figsize=(11, 3.3))
temp.loc["2014":"2016"].plot(ax=ax, alpha=0.35, linewidth=0.8, label="Daily")
trend_24.loc["2014":"2016"].plot(ax=ax, linewidth=2, label="24-day moving average")
ax.set(title="Raw series vs. moving average", xlabel="Date", ylabel="Temperature (°C)")
ax.legend()
plt.show()
```

Notice
- Raw data: short-term variation/noise
- Moving average: slower structure

  - Mean and median almost coincide, so no month is being dragged by a few extreme days.